

Homework 2

- Given the following bigram counts and probabilities from the Berkeley Restaurant Project corpus, compute the probability of the given sentences.

$$P(i \mid \langle s \rangle) = 0.25$$

$$P(\langle /s \rangle \mid \text{food}) = 0.68$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

- I want chinese food
- I want to eat chinese food
- I want to eat food

- Now, using add-1 smoothing, recalculate the probabilities.

$$P(i \mid \langle s \rangle) = 0.19$$

$$P(\langle /s \rangle \mid \text{food}) = 0.40$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

- Which of the two probabilities you computed in the previous exercise is higher, un-smoothed or smoothed? Explain why.
- Write a program to compute un-smoothed unigrams and bigrams.
 - Use any language you want, but do not use any libraries other than math/probability ones (Java.Math, numPy, etc)
 - Run your n-gram program on two different small corpora of your choice. Try and make them from different genres (i.e. a news article and a song lyric)
 - Now compare the statistics of the two corpora. What are the differences in the most common unigrams between the two? How about interesting differences in bigrams?
 - Add an option to your program to generate random sentences.

Homework 1

1. Write regular expressions for the following languages WITHOUT USING SHORTCUTS (i.e. no `\d`, `\b`, etc) (2.6.1 - 2.6.5)
 - a. the set of all alphabetic strings
 - b. the set of all lower case alphabetic strings ending in a *b*
 - a. Example matches: brb, hijab, b
 - b. Example non-matches: hello, ba
 - c. the set of all strings from the alphabet *a, b* such that each *a* is immediately preceded by and immediately followed by a *b*
 - a. Example matches: bbbbabbb, babab, bbbbbbabbbabbbbabab
 - b. Example non-matches: baab, aba
2. Write regular expressions for the following languages. By “word”, we mean an alphabetic string separated from other words by whitespace, any relevant punctuation, line breaks, and so forth. (2.6.6 - 2.6.9)
 - a. the set of all strings with two consecutive repeated words
 - a. Example matches: Humbert Humbert, the the
 - b. Example non-matches: big bug, the big bug
 - b. all strings that start at the beginning of the line with an integer and that end at the end of the line with a word
 - a. Example matches: 5 times I went to the Seven-Eleven
 - b. Example non-matches: 5 times I went to the 7-11
 - c. all strings that have both the word grotto and the word raven in them (but not, e.g., words like grottos that merely contain the word grotto)
 - a. Example matches: raven grotto, The raven flew over the grotto, grotto raven
 - b. Example non-matches: Quoth the raven grottos are cool!, raven’s grotto
 - d. write a pattern that places the first word of an English sentence in a register. Deal with punctuation.
3. Edit Distance (2.9.1)
 - a. Compute the edit distance (using insertion cost 1, deletion cost 1, substitution cost 2) of “leda” to “deal”. Show your work (using the edit distance grid).
 - b. Figure out whether `drive` is closer to `brief` or to `divers` and what the edit distance is to each.
 - c. Now implement a minimum edit distance algorithm in Java and use your hand-computed results to check your code.

Homework 3

Warmup:

Given the following short movie reviews, each labeled with a genre, either comedy or action:

1. fun, couple, love, love **comedy**
2. fast, furious, shoot **action**
3. couple, fly, fast, fun, fun **comedy**
4. furious, shoot, shoot, fun **action**
5. fly, fast, shoot, love **action**

and a new document D:

fast, couple, shoot, fly

compute the most likely class for D. Assume a naive Bayes classifier and use add-1 smoothing for the likelihoods.

Assignment:

Build a naive Bayes sentiment classifier that will assign reviews of an application as either **positive**, **neutral**, or **negative**.

- You will need to do some basic preprocessing on the documents (normalization, etc).
- Do not use a stop word list.
- Ignore any Out-Of-Vocabulary (OOV) terms when classfying

You are provided a small set of pre-classified training data to build your model. The data is formatted such that each line of text contains a document (the title of a review). The first token of each line will be the classification of that review, either POS, NEU, or NEG. Below is a sample document:

POS The program was quite helpful with creating websites.

An example output of your system may look something like this:

```
The program does what it should do. : POSITIVE
It functions adequately. : NEUTRAL
The program sucks. : NEGATIVE
This thing runs like a pregnant cow. : NEGATIVE
It was a little slow, but not too bad. : NEUTRAL
Slow. Slow. SLOW! : NEGATIVE
Great software! : POSITIVE
Worth the trouble to install. : NEUTRAL
```

Once the model has been built, feed in the provided test documents and write a report detailing your results. In the report, address the following:

- How accurate was the classifier? What was the Precision and Recall? The F-measure?
- Choose one incorrectly classified document.
 - Manually calculate the sentiment probabilities for the document (you can use your classifier to generate the likelihoods and prior probabilities, but do the classifying on paper)
 - What is the difference of the probability sums of the correct class and the class assigned by the system?
 - Identify the term or terms that caused the system to misclassify the document.
 - Build a document (or documents) to add to the training set that would allow the system to correctly classify the document.
 - Show the mathematical reasoning for your choice of words in the document.
 - Rerun the tests with the additional information.
 - Did adding the additional information change any other document classification? If so, how? Did it improve the overall accuracy of your system or make it worse?
- Add the MPQA Subjectivity Cues Lexicon to your system and run the tests again and report the results.
 - Choose a document that was classified differently after adding the lexicon. Was it correctly or incorrectly classified? Discuss why.
- Finally, use the provided collection of Amazon reviews from 2007 to train your classifier. Run the associated tests and report the Precision, Recall, and F-measure.
 - **Warning!** This is BIG and will take a long time to train. If your classifier is not efficient it could take over a day, maybe two in extreme cases. It will an hour or more on the best implementation. Plan accordingly (i.e. don't start this the night before it is due...)
- Briefly discuss what you learned from this assignment, what you liked or disliked about the assignment and, optionally, anything you would like to see changed or added to improve the assignment.